Adapting LLM Agents to Interactive Reasoning through Parallel Exploration

Yunfei Xie[1], Shiyi Lan[2], Chen Wei[1]

[1]Rice University , [2]NVIDIA

Abstract

ARC-AGI-3 is an interactive reasoning benchmark where agents must discover novel game mechanics and complete multi-level tasks through exploration, with no instructions provided. Frontier models fail to make meaningful progress, while humans solve the same games with ease. We attribute this gap to fundamentally novel mechanics that lie outside pretrained knowledge and extremely sparse rewards where only level completion provides feedback. To address these challenges, we propose two complementary techniques. Multi-turn RLVR adapts the model online by fine-tuning on level-completing trajectories. Level-synchronized parallel sampling deploys k agents in parallel; whenever any agent completes a level, all others synchronize to that agent's progress and advance together. Together, these techniques enable reaching levels the base model cannot while requiring up to 4× fewer actions. Models trained on ARC-AGI-3 also transfer to ALFWorld, nearly doubling baseline performance. Our project page is at https://yunfeixie233.github.io/ARC-AGI-3-RL/.

1. Introduction

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) [Chollet, 2019] measures whether AI systems can generalize to novel tasks rather than relying on narrow, pre-trained capabilities. Earlier versions [Chollet, 2019, Chollet et al., 2026] test static grid-based reasoning, where players deduce transformation rules from examples. ARC-AGI-3 [ARC Prize Foundation, 2026] shifts to interactive game environments that test *skill acquisition efficiency*: how quickly a system can learn new things when faced with novel situations requiring minimal prior knowledge. Agents are placed in games where they must discover goals and complete tasks through trial-and-error exploration over multiple steps. An agent that learns the game's rules faster will waste fewer actions on wrong attempts, so action efficiency serves as a direct proxy for learning efficiency [Kamradt, 2025a].

ARC-AGI-3 is extremely challenging for current AI systems. Frontier models, when placed in these novel environments without instructions, fail to explore effectively. They exhaust large action budgets without completing even basic levels, whereas humans solve the same games efficiently [Kamradt, 2025a,b]. In early evaluations, GPT-5 High Reasoning [OpenAI, 2025] exhibited repetitive, non-exploratory behavior on ls20, failing to complete even the first level [Kamradt, 2025a]. A subsequent 30-day agent competition found that frontier models including O3 High and Grok 4 similarly failed to make meaningful progress [Kamradt, 2025b,a].

What causes this gap? We identify two primary sources of difficulty. First, the tasks are fundamentally novel. Each ARC-AGI-3 game is built on distinct mechanics, and no instructions

or goals are provided to the agent [Kamradt, 2025b]. The games rely only on core knowledge priors, requiring the agent to adapt to unfamiliar environments through exploration alone [Chollet and Knoop, 2025]. Second, learning signals are extremely sparse. The only feedback ARC-AGI-3 provides is level completion itself, and most action sequences do not achieve it. Since each level typically requires tens of sequential correct actions, successful trajectories are exceedingly rare. Novel mechanics and sparse rewards together make ARC-AGI-3 exceptionally hard, exposing core difficulties in interactive reasoning over long-horizon tasks.

To address these challenges, we propose two complementary techniques. We introduce multi-turn Reinforcement Learning with Verifiable Rewards (RLVR) to handle novel game mechanics. The model adapts online by fine-tuning on level-completing trajectories, building game-specific strategies through experience. For sparse rewards, we observe that completing a level requires many sequential correct actions, so any single agent rarely succeeds. Running many agents in parallel dramatically increases the chance that at least one completes the level, and we share that progress across all agents. Whenever any agent finishes a level, all other agents synchronize to that agent's progress. This draws on the principle of pass@k, where multiple independent attempts yield a much higher success rate than any single try. We extend this principle from single-step generation to multi-step interactive tasks, formalized in Section 2, and detail the level-synchronized parallel sampling mechanism in Section 3.

Both components contribute meaningfully. On vc33, parallel sampling with level synchronization enables completion of all five levels, whereas the base model alone fails beyond Level 2. Adding RLVR further reduces the actions required to reach Level 5 from 79 to 19, approximately 4× fewer. On ft09, RLVR enables Level 3 completion where parallel sampling alone fails. Furthermore, policies learned on individual ARC-AGI-3 games generalize to ALFWorld [Shridhar et al.], a text-based household task benchmark with entirely different mechanics. The overall success rate improves from 14.8% to up to 28.5%, nearly doubling baseline performance, suggesting that the learned reasoning skills extend well beyond the training environment.

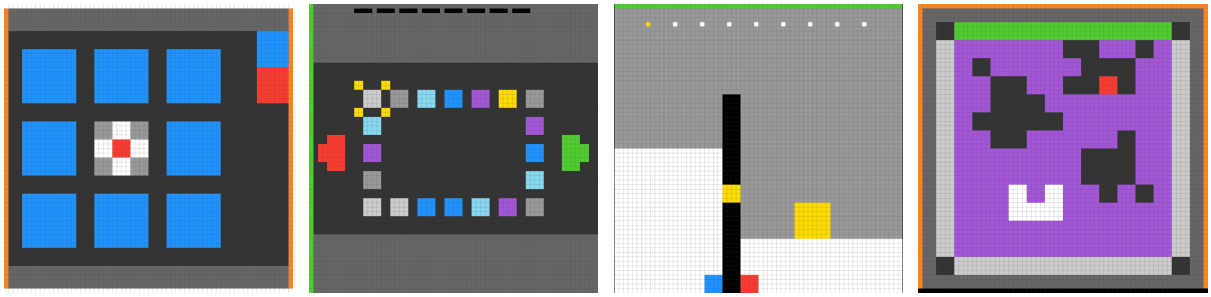
In summary, our contributions are:

- We propose a multi-turn RLVR framework that fine-tunes the model on level-completing trajectories collected during online interaction, enabling adaptation to novel game mechanics.
- We introduce level-synchronized parallel sampling, which deploys multiple agents in parallel and shares progress whenever any agent completes a level, substantially increasing level completion rates. This extends the pass@k principle to multi-step interactive tasks (Section 2).
- We demonstrate significant improvements on ARC-AGI-3 tasks, requiring up to 4× fewer actions to reach the same level, and show that learned capabilities transfer to ALFWorld, nearly doubling success rates.

2. Preliminaries

2.1. ARC-AGI-3 Overview

ARC-AGI-3 [ARC Prize Foundation, 2026] consists of 6 publicly available interactive game environments, each containing 8–10 levels of increasing difficulty, as illustrated in Figure 1. Unlike traditional benchmarks, ARC-AGI-3 provides no instructions or hints; agents must discover game mechanics, objectives, and winning conditions entirely through interaction. Further details on observation spaces, action spaces, and evaluation criteria are provided in Appendix A.



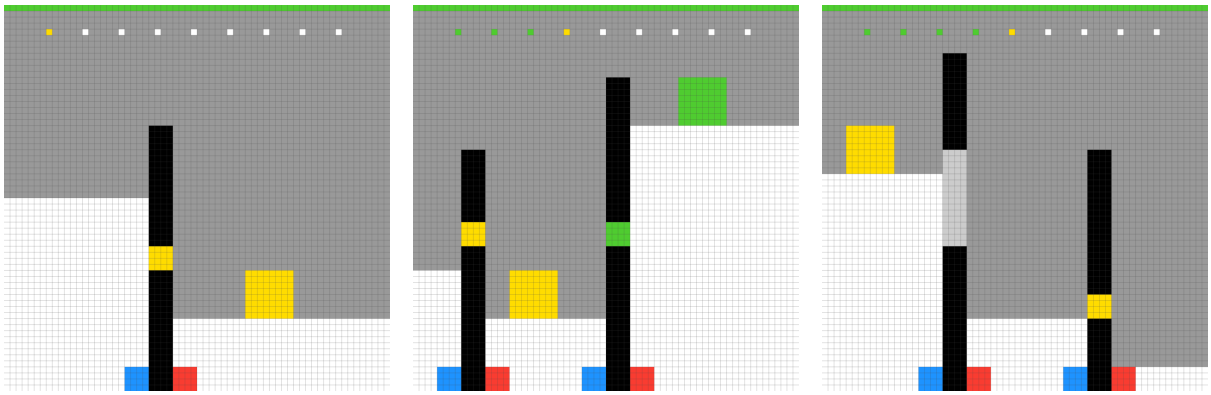
(a) ft09: Grid-based puzzle with spatial reasoning.

(b) lp85: Board-like environment with colored pieces.

(c) vc33: Orchestration game with liquid level coordination.

(d) as66: Shape manipulation on a purple canvas.

Figure 1 | **Example game environments from ARC-AGI-3.** Each game presents unique interactive reasoning challenges requiring agents to learn game mechanics through exploration without instructions. Agents must discover rules, plan actions, and complete multiple levels of increasing difficulty.



(a) Level 1: Single block, single target.

(b) Level 4: Two blocks, two targets.

(c) Level 5: Conditional target activation.

Figure 2 | **Progressive level design in vc33.** Players coordinate liquid levels to position floating blocks at target heights. **Level 1** (left): adjust one chamber to raise a yellow block to its target marker. **Level 4** (middle): coordinate two independent chambers to align two blocks with their respective targets. **Level 5** (right): a gray validation zone requires both liquid levels to fall within a shared region simultaneously, adding conditional logic. Each level retains earlier mechanics while introducing new elements.

Progressive Level Design. Within each game, levels progressively add new elements while retaining earlier ones, as illustrated for vc33 in Figure 2. Early levels serve as tutorials that introduce one mechanic at a time, while later levels layer additional constraints that require integrating cumulative knowledge [Kamradt, 2025a].

2.2. Problem Setup

We consider a setting in which an LLM agent interacts with an ARC-AGI-3 environment to complete multi-step tasks. Each *game* (e.g., ft09, lp85, vc33) consists of a sequence of *levels* with increasing difficulty, indexed by $\ell = 1, 2, \ldots, L$. At each discrete time step $t = 1, 2, \ldots$, the agent observes a state $\boldsymbol{s}_t \in \mathcal{S}$ representing the current game screen and selects an action $a_t \in \mathcal{A}$ from a finite discrete action space. The environment then transitions to the next state $\boldsymbol{s}_{t+1}$ and provides a sparse reward $r_t \in \{0, 1\}$ indicating whether the agent has completed the current level.

A *trajectory* $\boldsymbol{\tau} = \{(\boldsymbol{s}_1, a_1, r_1), (\boldsymbol{s}_2, a_2, r_2), \ldots, (\boldsymbol{s}_T, a_T, r_T)\}$ records the full sequence of T interactions within an episode. The agent's behavior is governed by a policy $\pi_\theta(a_t|\boldsymbol{s}_t)$, parameterized by θ, which defines a distribution over actions conditioned on the current state. The objective is to complete all L levels within a step budget of N steps.

Unlike settings with dense per-step rewards, ARC-AGI-3 provides only sparse, verifiable feedback at level completion. As discussed in Section 1, this challenge grows as levels become longer, making level completion increasingly unlikely.

2.3. Single-Step Pass@k

The pass@k metric was originally introduced for evaluating code generation models [Kulal et al., 2019, Chen et al., 2021]. In this *single-step* setting, the model generates k candidate outputs for a given input, and a problem is considered solved if *any* of the k samples passes verification. The unbiased estimator is

$$\text{pass@}k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \tag{1}$$

where n is the total number of samples generated, and c is the number of correct samples.

2.4. Multi-Step Pass@k

We extend pass@k to *multi-step* agentic tasks where level completion requires a sequence of actions rather than a single generation.

For multi-step tasks, given a maximum step budget N per episode, we run k independent episodes on the same task. A level is considered completed if at least one of the k episodes completes it. Formally,

$$\text{pass@}k := \mathbb{E}_{\text{Levels}} \left[\mathbf{1} \left[\exists\, i \in [k] : S_i \leq N \right] \right] \tag{2}$$

where S_i denotes the number of steps for episode i to complete the task, with $S_i = \infty$ if it fails, and $[k] = \{1, \ldots, k\}$.

The key difference from single-step pass@k is that each "sample" is now an entire trajectory of actions, not a single output. We refer to this approach of deploying k agents in parallel as *parallel sampling*, and describe our level-synchronized implementation in Section 3.

3. Method

Our approach has two main components. Level-synchronized parallel sampling deploys multiple agents and shares progress whenever any one completes a level. Multi-turn RLVR with trajectory chunking improves action selection over successive rounds of interaction.

3.1. Level-Synchronized Parallel Sampling

From Sparse Rewards to Parallel Sampling. As discussed in Section 1, the sparse reward structure of ARC-AGI-3 makes any single trajectory unlikely to complete a level, since success demands a long sequence of correct actions with no intermediate positive feedback. Deploying multiple independent agents increases the probability that at least one succeeds, drawing on the pass@k principle that multiple independent attempts yield higher success rates (Section 2). The key insight is to synchronize at the granularity of each level rather than the entire game. Whenever any agent completes a level, all other agents synchronize to that agent's progress and move on to the next level together.

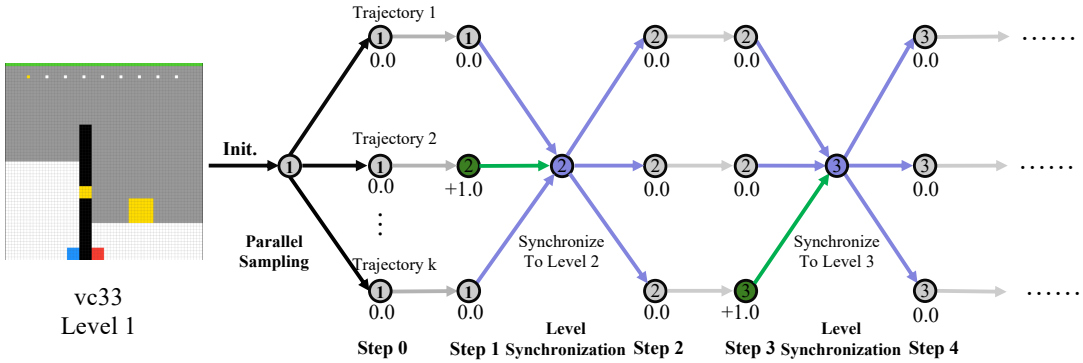


Figure 3 | **Level-synchronized parallel sampling with test-time training on VC33.** We initialize k parallel agents identically at Level 1. Each training step consists of C game actions; a green node with +1.0 reward indicates level completion, while gray nodes with 0.0 reward indicate no completion. The number inside each node denotes the agent's current level. Thanks to multi-step pass@k, running k parallel agents greatly increases the chance that at least one completes the current level. Once one agent completes the level, Level Synchronization shares that progress with all agents, as shown by the purple hub nodes and purple fan-out arrows, so that every agent resumes exploration from the newly reached level rather than starting over.

Level Synchronization. As illustrated in Figure 3, we instantiate k parallel agent instances, each operating in its own environment initialized at the first level. Each agent independently attempts to complete the current level through its own sequence of actions. When any agent completes the current level, all other agents synchronize to that agent's state and advance together to the next level, where all k agents resume independent exploration. This process repeats until all levels are completed or the step budget is exhausted.

This synchronization mechanism directly addresses the sparse reward problem by improving sampling efficiency. Deploying k agents in parallel increases the probability that at least one discovers a level-completing action sequence. Once any agent completes the current level, all other agents synchronize to that agent's state rather than starting from scratch. The k agents collectively need to find only one solution per level instead of each agent independently solving all levels from scratch. Note that agents do not carry forward an explicit conversation history or memory buffer across levels; each agent begins a new level from the synchronized initial observation.

3.2. Trajectory Chunking

Many levels in ARC-AGI-3 require dozens of attempts to complete, with individual episodes spanning 100 or more steps. Running until level completion in a single training step is problematic for two reasons. First, the agent may execute a long sequence of actions without receiving any positive feedback. Second, even when the level is eventually completed, attributing a single reward across a very long action sequence makes credit assignment difficult.

To address this, we introduce *trajectory chunking*. Rather than running until level completion before assigning a reward, we divide each trajectory into segments of at most C actions and evaluate whether the level was completed after each segment. The agent executes up to C actions starting from the current state $\boldsymbol{s}_t$ and receives a reward at the end of each segment. As illustrated in Figure 3, each node represents one such segment. A green node with reward +1.0 indicates that the level was completed within that segment, while a gray node with reward 0.0 indicates that it was not. When the level is not completed, the next segment resumes from the resulting state $\boldsymbol{s}_{t+C}$. This bounds the length of each training trajectory, so that when a positive

reward is obtained, it can be meaningfully attributed to the corresponding actions that lead to level completion. The specific values of k and C are detailed in Section 4.1.

3.3. Overall Pipeline

The two components form an iterative loop between exploration and learning. Multiple agents explore the environment in parallel, and whenever any agent completes a level, that progress is shared with all others. The successful trajectories are then used to update the policy, which produces a stronger set of agents for the next round of exploration. Over successive rounds, the agents become increasingly effective at discovering level-completing strategies, enabling progress on levels that were initially out of reach.

4. Experiments

4.1. Experiment Setup

Games. We evaluate our approach on four of the six publicly available ARC-AGI-3 games: ft09, lp85, vc33, and as66, as shown in Figure 1. These four games cover all three control schemes (click-based, arrow-based, and combined) and span a range of difficulty levels. Each game requires the agent to discover game mechanics and execute multi-step action sequences to complete levels.

Training details. We use Qwen2.5-7B-Instruct [Yang et al., 2024] as our base model and perform full-parameter fine-tuning with FSDP, using a learning rate of 1×10^{-6}. We deploy $k = 128$ parallel agents with level synchronization. Training is conducted in an *online* setting, where the model interleaves policy updates with environment interaction for each game.

For policy optimization, we adopt GRPO [Shao et al., 2024]. In each training round, the k parallel agents produce trajectory segments (each of at most C actions). We group every 8 trajectory segments together for advantage computation, yielding groups of $G = 8$ segments $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_G\}$. Each segment $\boldsymbol{\tau}_i$ receives a binary reward $R(\boldsymbol{\tau}_i) \in \{0, 1\}$ indicating whether the level was completed within that segment. Following GRPO, we compute the advantage for each segment by normalizing its reward with respect to the group statistics.

$$\hat{A}(\boldsymbol{\tau}_i) = \frac{R(\boldsymbol{\tau}_i) - \mathrm{mean}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^{G}\big)}{\mathrm{std}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^{G}\big)}. \tag{3}$$

The policy is then updated via the clipped surrogate objective with KL regularization.

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} \min\Big(\rho_\theta(\boldsymbol{\tau}_i)\,\hat{A}(\boldsymbol{\tau}_i),\ \mathrm{clip}\big(\rho_\theta(\boldsymbol{\tau}_i), 1 \pm \epsilon\big)\,\hat{A}(\boldsymbol{\tau}_i)\Big)\right] - \beta\,\mathbb{D}_{\mathrm{KL}}\big(\pi_\theta \,\|\, \pi_{\mathrm{ref}}\big), \tag{4}$$

where $\rho_\theta(\boldsymbol{\tau}_i) = \pi_\theta(\boldsymbol{\tau}_i)/\pi_{\theta_{\mathrm{old}}}(\boldsymbol{\tau}_i)$ is the importance sampling ratio. We set the KL coefficient $\beta = 0.01$ and the discount factor $\gamma = 0.95$. The trajectory chunk size C is set to 5, limiting each training segment to at most 5 actions. Training runs for 150 epochs on 8×A100 GPUs. The prompt template used for agent-environment interaction is provided in Appendix C.

Metrics. We evaluate performance by the cumulative number of actions required to reach each level, the official ARC-AGI-3 evaluation criterion [Kamradt, 2025a]. Agents are scored primarily on levels completed, with action count as the tiebreaker. Cumulative actions capture both whether a level is reached and how quickly.

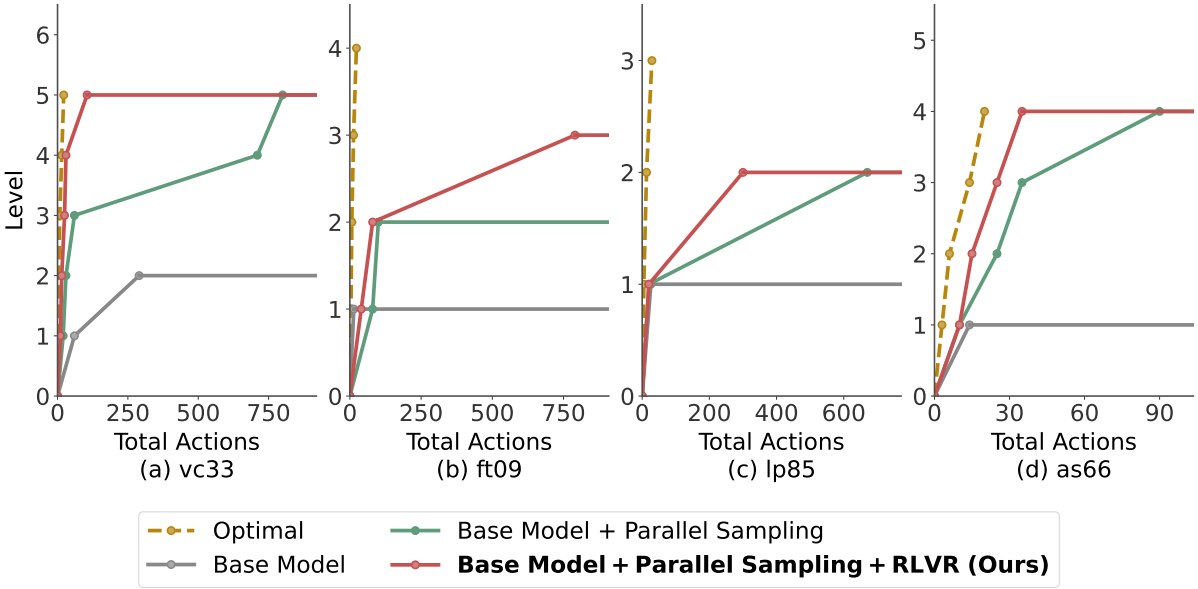


Figure 4 | **Cumulative actions required to reach each level.** For each game, the x-axis shows the total number of actions from game start and the y-axis shows the level reached. Steeper curves indicate faster level completion, meaning the method reaches the same level with fewer actions. We compare four settings. The base model alone, the base model with parallel sampling, our full method combining parallel sampling with multi-turn RLVR, and the optimal solution, which solves each game using the minimum number of actions.

4.2. Results

Parallel sampling and multi-turn RLVR both contribute to level completion. Figure 4 presents the cumulative actions required to reach each level across the four evaluated games, with detailed results in Table 2. The base model alone completes early levels but fails to advance beyond the first or second level on most games. Adding parallel sampling substantially increases the levels reached. For example, on vc33, parallel sampling enables completion of all five levels, reaching Level 5 in 79 actions. Adding RLVR on top of parallel sampling further reduces the actions required to reach Level 5 from 79 to 19, an approximately 4× reduction. On ft09, RLVR enables Level 3 completion at 78 actions, a level that parallel sampling alone cannot reach. These results show that the two components provide complementary benefits. Parallel sampling extends the set of reachable levels, while RLVR improves action selection within those levels.

Our method generalizes across diverse game mechanics. The four games exhibit different difficulty characteristics and control schemes. vc33 uses click-based control with an action space of 4,096 possible click locations. lp85 also uses click-based control but presents a more challenging exploration landscape, where even parallel sampling struggles beyond Level 2. ft09 combines spatial reasoning with sequential dependencies, making later levels particularly difficult. as66 uses a combined control scheme integrating both arrow and click inputs. Despite these variations, our method consistently improves performance across all games. Parallel sampling provides large gains on games where the base model struggles, and RLVR further refines action selection to reduce the total actions required.

Learned capabilities transfer beyond ARC-AGI-3. A natural question is whether the trained models pick up general reasoning skills or simply memorize game-specific patterns. We test this

Table 1 | **Transfer to ALFWorld.** Models trained on individual ARC-AGI-3 games are evaluated on ALFWorld [Shridhar et al.] to test whether learned capabilities generalize beyond the training game.

Model	ALFWorld						
	Pick	Look	Clean	Heat	Cool	Pick2	All
Baseline	33.4	21.6	19.3	6.9	2.8	3.2	14.8
Ours (ft09)	60.6	43.6	**34.4**	6.3	9.4	0.0	**28.5**
Ours (lp85)	44.3	47.8	27.2	3.1	5.6	3.1	22.7
Ours (vc33)	**60.7**	41.7	20.0	**21.4**	**11.1**	4.5	26.6
Ours (as66)	50.9	**55.8**	26.4	19.6	9.3	**5.4**	27.7

by evaluating transfer to ALFWorld [Shridhar et al.], a text-based household task benchmark with fundamentally different mechanics and observation modalities. As shown in Table 1, all four trained models substantially outperform the baseline on the overall ALFWorld success rate, with improvements ranging from 7.9 to 13.7 percentage points. The transfer patterns differ by game. ft09 excels at Clean tasks (34.4%), vc33 shows strong performance on Heat (21.4%) and Cool (11.1%) tasks, and as66 achieves the highest Look accuracy (55.8%). These complementary strengths suggest that each game teaches distinct reasoning skills that transfer selectively. The consistent gains across all four models indicate that the learned policies capture general interactive reasoning strategies rather than any single game's mechanics.

5. Related Work

Reinforcement Learning for LLM Agents. Reinforcement learning has become a central approach for training LLM agents in interactive environments. Early work applied classical algorithms such as DQN [Van Hasselt et al., 2016] to text-based games [Narasimhan et al., 2015], while later approaches adopted policy gradient methods like PPO [Schulman et al., 2017] for embodied tasks [Shridhar et al.] and web-based environments. More recently, several frameworks have addressed multi-turn RL for LLM agents. RAGEN [Wang et al., 2025] proposes StarPO, a trajectory-level GRPO variant that concatenates states, reasoning, and actions into episode-level responses, but faces scalability challenges in long-horizon settings. Search-R1 [Jin et al., 2025] trains LLMs to use search engines during multi-step reasoning via outcome-based rewards and retrieved token masking. LOOP [Chen et al., 2025] combines RLOO with PPO-style updates for long-horizon interactive agents. AgentGym-RL [Xi et al., 2025] introduces ScalingInter-RL for balancing exploration and exploitation across diverse tasks. Our work differs from these frameworks in two ways. We operate in visual environments with fundamentally novel mechanics (no text-based instructions), and we introduce a level-synchronized parallel sampling mechanism that shares progress across agents rather than training each agent independently.

Test-Time Scaling and Parallel Search. Parallel sampling at test time has been widely studied in code generation through the pass@k framework [Chen et al., 2021], where generating k independent samples increases the probability of producing a correct solution. Recent work has extended test-time compute scaling to reasoning tasks through methods such as Monte Carlo Tree Search (MCTS) [Silver et al., 2017], best-of-n sampling, and iterative self-refinement. However, these methods are primarily designed for single-turn generation and do not directly apply to multi-step interactive tasks where partial progress (e.g., completing intermediate levels) can be shared across agents. Our level-synchronized parallel sampling extends pass@k to the multi-step setting, providing a mechanism for checkpointing and sharing progress that is

complementary to single-turn search methods.

6. Conclusion

We have presented an approach for solving ARC-AGI-3, an interactive reasoning benchmark that poses fundamental challenges for current AI systems due to novel game mechanics and sparse learning signals. The two core techniques, multi-turn RLVR and level-synchronized parallel sampling, work together to reach levels the base model cannot while requiring up to 4× fewer actions. Transfer results on ALFWorld further show that the learned skills are not confined to the training games, pointing toward agents that can efficiently adapt to novel tasks through exploration and sequential decision-making.

Acknowledgments

We would like to thank Greg Kamradt[1] and the ARC Prize Foundation[2] for their help. We also thank Lambda Inc.[3] for providing a research grant that supported the computational resources used in this work.

References

ARC Prize Foundation. ARC-AGI-3: Interactive reasoning benchmark. https://arcprize.org/arc-agi/3/, 2026. Accessed: 2026.

Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

François Chollet and Mike Knoop. Francois chollet + mike knoop: Fireside chat. ARC Prize @ MIT, 2025. Video available at https://www.youtube.com/watch?v=RsmSkmGjHso.

Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2026. URL https://arxiv.org/abs/2505.11831.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Greg Kamradt. ARC-AGI-3 and action efficiency. ARC Prize @ MIT, 2025a. Video available at https://www.youtube.com/watch?v=bqNfIHedb3g.

Greg Kamradt. Interactive reasoning benchmarks: ARC-AGI-3 preview. ARC Prize @ MIT, 2025b. Video available at https://www.youtube.com/watch?v=3T4OwBp6d90.

[1] https://gregkamradt.com/
[2] https://arcprize.org/
[3] https://lambda.ai/

Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. Spoc: Search-based pseudocode to code. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper/2019/file/7298332f04ac004a0ca44cc69ecf6f6b-Paper.pdf.

Karthik Narasimhan, Tejas Kulkarni, and Regina Barzilay. Language understanding for text-based games using deep reinforcement learning. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1–11, 2015.

OpenAI. Gpt-5 system card. *openai.com/index/gpt-5-system-card*, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Yu Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.

Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, et al. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

A. ARC-AGI-3 Details

This appendix provides additional details on the ARC-AGI-3 benchmark, including the evaluation protocol, observation format, and action spaces.

Evaluation. The benchmark's evaluation criterion prioritizes both effectiveness and efficiency. Agents are scored primarily on the number of levels completed, with fewer total actions used as a tiebreaker among equally performing agents. This dual objective encourages solutions that not only discover winning strategies but do so with minimal exploration.

Interaction Protocol. Agents interact with the environment through a step-based API. At each step, the agent receives a visual observation and submits an action, then receives the next observation along with a binary signal indicating whether the current level was completed. Levels within each game must be completed sequentially. If the agent satisfies the winning conditions, the environment advances to the next level; otherwise, the current level resets to its initial state when the step limit expires.

Visual Observations. Agents receive observations as 64×64 pixel RGB frames with a discrete palette of 16 colors. Each frame contains both the game environment and a status bar displaying the number of steps remaining before an automatic level restart. When the step counter reaches zero, the current level resets to its initial state. In the majority of games, the number of levels passed is also displayed.

Action Spaces. The benchmark features three control schemes. Games such as `ls20` use arrow-based control with directional keyboard inputs (up, down, left, right), yielding an action space of size $|\mathcal{A}| = 4$. Games such as `ft09`, `vc33`, and `lp85` use click-based control, allowing the agent to click any pixel location in the frame, yielding $|\mathcal{A}| = 64 \times 64 = 4{,}096$. Games such as `sp80` and `as66` use combined control schemes that integrate both arrow and click inputs, resulting in $|\mathcal{A}| = 4{,}100$. These large action spaces, combined with the long trajectories required for level completion, make successful action sequences combinatorially unlikely and reinforce the sparse reward structure described in Section 1.

B. Additional Experimental Results

Model	Level 1	Level 2	Level 3	Level 4	Level 5
vc33					
Optimal	2	4	8	15	22
Base Model	5	28	>1000	>1000	>1000
Base Model + Parallel Sampling	1	2	5	70	79
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**0**	**1**	**4**	**15**	**19**
ft09					
Optimal	4	7	13	23	33
Base Model	13	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	7	9	>150	>150	>150
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**3**	**7**	**78**	>150	>150
lp85					
Optimal	5	13	29	>29	>29
Base Model	25	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	**1**	66	>150	>150	>150
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**1**	**29**	>150	>150	>150
as66					
Optimal	3	6	14	20	>150
Base Model	13	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	**1**	4	6	17	>150
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**1**	**2**	**4**	**6**	>150

Table 2 | **Cumulative actions required to reach each level on ARC-AGI-3 games.** We compare the optimal solution that solves each game with the fewest possible actions, the base model alone, the base model with parallel sampling, and the full method combining parallel sampling with multi-turn RLVR (Ours). Lower values indicate faster level completion. **Bold** numbers indicate the best model performance in each column within each game.

C. Prompt Template

Prompt Template for ARC-AGI-3

 # CONTEXT
You are an agent playing a dynamic game. Your objective is to WIN and avoid GAME_OVER while minimizing actions.

One action produces one Frame. One Frame is made of one or more sequential Grids. Each Grid is a matrix of size 64×64, each cell represents a visual component of the game.

 # HISTORY
Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} actions you took:
{action_history}

 # CURRENT STEP
You are now at step {current_step}.

 # CURRENT GRID
<image>

 # GAME STATE
Current score: {score}

 # INTERACTIVE COMPONENTS
{component_context}

 # TURN
Based on your action history, the current grid, and interactive components, choose one interactive component to interact with to advance toward your game objective.

 # ACTION FORMAT
You must output ONLY the coordinates where you want to click, in the format (x, y) where:
- x and y are integers between 0 and 63 (inclusive)
- The grid is 0-indexed, so (0, 0) is top-left and (63, 63) is bottom-right
- Coordinates outside this range are invalid and will cause errors
Do not include any other text, just the coordinates.

Example response:
<think>
[Your detailed analysis of your previous actions, the current board state, review of interactive components and their positions, review of game state, and choose one interactive component to interact with to advance toward your game objective.
</think>
<answer>(4, 32)</answer>

Figure 5 | Prompt template used for the LLM agent interacting with ARC-AGI-3 environments. Template variables are shown in brown. The agent outputs its reasoning within <think> tags and its action within <answer> tags.